M1 — Literature survey, agent design and project plan

Name: Leonie Merkl **Team:** A Team **Specialisation:** Musicologist (Research)

1 Literature survey

Retrieval. Retrieval-augmented generation (RAG) first retrieves passages for the input and then generates the answer from them. The model itself does not decide whether or when to retrieve [1]. Its dense retriever, DPR, was better than BM25 on all datasets tested except SQuAD [2]. At the Reading Desk, only the titles can be searched for free, so retrieval starts from them.
Agentic reading. ReAct alternates reasoning and tool calls. Non-informative searches caused 23% of its errors, and it often repeated earlier thoughts and actions [3]. Unlike RAG that retrieves once or follows a predefined workflow, A-RAG lets the model choose among keyword search, semantic search and chunk reading, and it was more accurate than traditional RAG with comparable or fewer retrieved tokens [4]. DeepRead shows the model the document's headings and lets it choose between searching and reading. It mostly searched first and then read [5]. At the Reading Desk, each read is charged in characters as well as tokens.
Cost and context. In Wu et al., each piece of evidence has a price and each question a budget to spend on it. No fixed selector uniformly dominated, and one model often consumed its budget instead of managing it [6]. Joren et al. found that raising the context limit from 6,000 to 10,000 tokens left the share of questions with sufficient context effectively unchanged [7]. The Reading Desk sets no budget itself and only counts the cost against the score.
Evaluation. Kapoor et al. argue that agents should be judged on accuracy and cost together, as a Pareto front: on HumanEval, simple baselines matched or beat complex agents at much lower cost [8]. ALCE tests whether citations jointly support each statement and whether any citation is irrelevant [9].

In a workflow, models and tools follow predefined code paths, whereas in an agent the model directs its own process and tool use [10]. By this definition, classic RAG is a workflow, while A-RAG and DeepRead are agents. The Reading Desk combines agentic reading, a charge per read and a score on the exact supporting set. No source in this survey treats all three together, so the design draws on each strand: term matching, agentic reading, a budget in code and cost-aware evaluation.

2 Agent design

2.1 My agent

The problem. Only `browse`, which lists titles and lengths, is free. `examine` (abstract and section headings) and `read` (one section) cost characters, even when repeated. A right answer scores 1, a wrong one or a skip 0. The named volumes score 1 if they are exactly a set that establishes the answer. An extra volume takes off a share, and a missing one gives 0. Extra reading and tokens also count, with weights published before M2 [11]. If the weights allow, no question will be skipped, and one volume too many will be named rather than one too few.
Approach. The *plain-prompt baseline* gives the model only the rules and the arena's messages. It plays alone with the arena's queries, without help from code and with a fixed limit per question [12]. It is the reference point and will be built first. The agent then adds classical computation in code: `bm25_search`, `find_in_toc` and regex. BM25 costs nothing, and term matching is sensitive to highly selective keywords [2], which names, titles of works and years are expected to be. Dense retrieval is left out for now because titles are short, but it could be added later.

The *agentic approach* (Fig. 1) is the main design. The titles from `browse` are in the prompt, and the model chooses every tool in a ReAct loop [3]. As in A-RAG, no fixed search step comes before its first decision [4]. The model can therefore react to what it finds: it can open a second volume when one is not enough, or search again when BM25 misses a synonym. Because each step costs tokens, the code guards the loop against the repetition reported for ReAct [3]. As seen in Wu et al., a model may use up its budget instead of managing it [6], so the budget is enforced in code. When the loop stops, the best guess is sent instead of a skip [11].

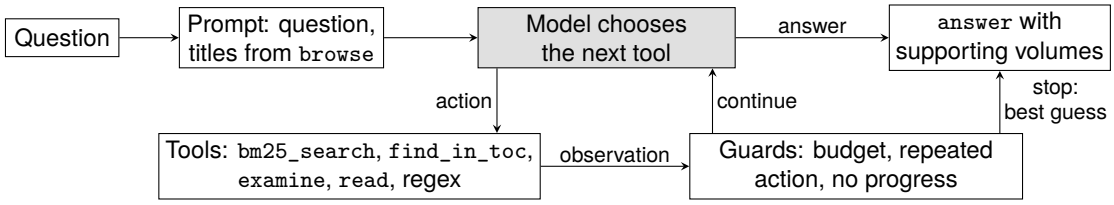

Figure 1: Agentic approach: the model decides each step; code runs tools and guards the loop.

The *workflow approach* is the alternative considered. Like classic RAG (§1), it follows a path set by code: it searches the titles, examines the top volumes, reads sections and calls the model only when checks in code are unclear. It would spend fewer tokens, but was not chosen because a fixed path cannot react when a question needs a second volume or a title uses a synonym.

How it will be judged. Correct answers, characters read and tokens will be measured, and the named volumes will be checked by citation recall and precision [9]. Runs will use questions kept out of tuning, several seeds and a fixed temperature [13], [14]. The agent works if it answers more questions correctly than the baseline within the budget per question. This budget starts at about 6,000 tokens [7], estimated here as 24,000 characters. The Pareto front of cost and correct answers will then be used to choose between settings [8]. If the agent falls short, each wrong answer will be traced to its cause: either the right section was never read, or it was read and the answer was still wrong. The more frequent cause will be fixed first.

Where it meets the team. The Musicologist gets a question and a budget from the Conductor and returns an answer, supporting volumes, a confidence and the cost, which the Conductor checks. Its model calls go through the shared wrapper. The shared store keeps the Musicologist's questions, the sections it has read and its best answer so far. For a new question, the sections already read are searched first, as a repeated read is charged again [11]. If the Musicologist fails, the Conductor can send the stored answer instead of a skip. In the Overture, the Musicologist rates signs and volumes by BM25 against open questions, without tokens (§2.2).

2.2 The Ensemble

Dependencies. The parts depend on each other in the four ways named by Malone and Crowston [15]. There are two *shared resources*: the places on the trolley and the one connection to the arena. As *producer and consumer*, the Improviser delivers the volumes that the Musicologist can read only in the hall, and a door opened by the Player gives the Improviser a shortcut. *Simultaneity* is needed at a door, where a puzzle can be answered only while the trolley stands there [11]. Each question is a *task with subtasks*: choosing rooms, fetching, reading and answering.

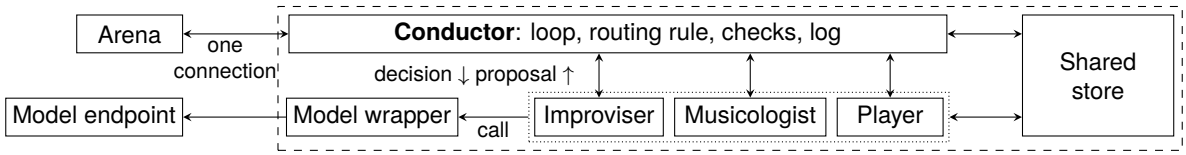

Figure 2: The container (dashed) with its three specialists (dotted). The Conductor and the specialists use the shared store. Only the specialists call the model, through the wrapper.

Coordination. The Ensemble is organised as a hierarchy [16]. At the top is the Conductor,

written in plain code. It holds the connection, runs the loop, checks every action and hands each decision to one specialist (Fig. 2). It also keeps the shared store: the setlist texts, sent only once, the map, the texts read and the costs. In a base quest, the first message names the quest and so the specialist. In the Overture the order of work is open, so a fixed rule settles where the trolley goes next: the Musicologist rates each candidate sign or volume, the Improviser gives its distance, and the Conductor takes the candidate with the most worth per distance.

The alternative considered was a blackboard [17], on which a control component selects which part writes next [16]. Its authors noted that such control "takes time and requires many fairly sophisticated mechanisms" and is justified where no explicit algorithm exists [17, p. 246]. Here a simple rule chooses the next step without using tokens, so only the shared store was taken over.

When a part fails, is slow or is wrong. If a specialist fails, the Conductor falls back on a default without tokens. For a question, it sends the best answer so far, which the Musicologist keeps up to date in the shared store. A puzzle is skipped, and on the map the nearest exit is taken. A slow model call is stopped before the endpoint's 300-second cut-off [12] and retried once, so a turn stays below the ten minutes of silence that end a run [11]. Against wrong answers, every proposal is first checked in code, as language models were found no better at verifying solutions than at producing them [18]. A failed proposal is returned once with the reason. If it fails again, the last well-formed one is sent, as a wrong answer costs no more than a skip [11], except on claims, which a team of three does not get.

Logging. The starter code logs every arena message and model call [12]. For each action, the log will add the deciding part, its reason, its tokens, the change in `spent` and the check result.

3 Project plan

3.1 The team's plan

By	What will be working	Who
16 Nov	Self-test passes, and fails when the prompt is broken	All
20 Nov	Conductor shares: loop and routing (Jurcik), shared store (Merkl), logging and failure rules (Mukherjee)	one each
23 Nov	Each base quest plays a practice run via the Conductor	each owner
27 Nov	**First Overture run with all parts in one container**	All
30 Nov week	Rehearsal: image connects to course arena and endpoint	Jurcik
4 Dec	Overture end to end with failure rules, baselines measured	Mukherjee
8 Dec	Image frozen, all dependencies inside, reflections drafted	Merkl
10 Dec	M2 handed in	each

Each member owns one Conductor share. Open rows are checked weekly and work is moved.

3.2 My plan

By	What will be working
20 Nov	Shared store with the best-answer field, baseline runs
23 Nov	Tools, practice run via the Conductor
27 Nov	Agentic approach with guards, ratings of signs and volumes
1 Dec	Measuring script, comparison with the baseline on held-out questions
7 Dec	Tuning along the Pareto front, settings fixed

Tuning is the step most likely to slip, and it would then shrink to a small grid search.

References

[1] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33*, Curran Associates, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.

[2] V. Karpukhin, B. Oğuz, S. Min, *et al.*, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550.

[3] S. Yao, J. Zhao, D. Yu, *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR 2023)*, 2023. arXiv: 2210.03629. [Online]. Available: https://arxiv.org/abs/2210.03629v3.

[4] M. Du, B. Xu, C. Zhu, *et al.*, *A-RAG: Scaling agentic retrieval-augmented generation via hierarchical retrieval interfaces*, 2026. arXiv: 2602.03442.

[5] Z. Li, H. Tian, L. Luo, Y. Cao, and P. Luo, *DeepRead: Document structure-aware reasoning to enhance agentic search*, Preprint, 2026. arXiv: 2602.05014.

[6] M. Wu, H. Yang, O. Ben-Porat, and Y. Ziser, *When knowledge is not free: Cost-aware evidence selection in retrieval-augmented generation*, 2026. arXiv: 2606.02245.

[7] H. Joren, J. Zhang, C.-S. Ferng, D.-C. Juan, A. Taly, and C. Rashtchian, "Sufficient context: A new lens on retrieval augmented generation systems," in *International Conference on Learning Representations (ICLR 2025)*, 2025. arXiv: 2411.06037.

[8] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "AI agents that matter," *Transactions on Machine Learning Research*, 2025. arXiv: 2407.01502. [Online]. Available: https://openreview.net/forum?id=Zy4uFzMviZ.

[9] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, 2023. arXiv: 2305.14627.

[10] Anthropic, *Building effective agents*, Anthropic Engineering, Dec. 2024. [Online]. Available: https://www.anthropic.com/engineering/building-effective-agents.

[11] IN2106 Multi-Agent AI Lab, *The Archive Games: Rules and API reference*, Course document, IN2106 Multi-Agent AI Lab, Technical University of Munich, Nov. 9, 2026.

[12] J. Schaumeier, *The Archive Games*, Lecture slides, Session 4, IN2106 Multi-Agent AI Lab, Technical University of Munich, Nov. 9, 2026.

[13] A. Hochlehnert, H. Bhatnagar, V. Udandarao, S. Albanie, A. Prabhu, and M. Bethge, "A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility," in *Second Conference on Language Modeling (COLM 2025)*, 2025. arXiv: 2504.07086. [Online]. Available: https://openreview.net/forum?id=90UrTTxp50.

[14] J. Schaumeier, *Language models and classical methods*, Lecture slides, Session 3, IN2106 Multi-Agent AI Lab, Technical University of Munich, Nov. 2, 2026.

[15] T. W. Malone and K. Crowston, "The interdisciplinary study of coordination," *ACM Computing Surveys*, vol. 26, no. 1, pp. 87–119, 1994. DOI: 10.1145/174666.174668.

[16] J. Schaumeier, *How do several agents decide together?* Lecture slides, Session 2, IN2106 Multi-Agent AI Lab, Technical University of Munich, Oct. 26, 2026.

[17] L. D. Erman, F. Hayes-Roth, V. R. Lesser, and D. R. Reddy, "The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty," *ACM Computing Surveys*, vol. 12, no. 2, pp. 213–253, 1980. DOI: 10.1145/356810.356816.

[18] S. Kambhampati, K. Valmeekam, L. Guan, *et al.*, “Position: LLMs can’t plan, but can help planning in LLM-Modulo frameworks,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, PMLR, 2024, pp. 22 895–22 907. arXiv: 2402.01817. [Online]. Available: https://proceedings.mlr.press/v235/kambhampati24a.html.

Use of AI

Claude Opus 5.5 and Claude Fable 5.2 (Anthropic) were used to summarise papers, to work out the design in conversation, to draft and edit the text and to produce the process diagram. The papers were read by the author, and every source, quotation and number cited from them was checked against the original. The design choices are the author’s own, and the final text and figure were checked against the author’s notes and the API reference.